

École Nationale des Sciences Appliquées
ENSA Béni Mellal

Filière : Transformation Digitale Industrielle

CAHIER DES CHARGES

Mini-Projet d'Intelligence Artificielle

LumiQuality AI

Système intelligent de prédiction de la qualité dans la fabrication de pièces plastiques par injection moulage

Application web full-stack basée sur l'IA pour l'industrie 4.0

Année universitaire 2025-2026

1. Présentation du projet

1.1 Identification du projet

Champ	Valeur
Nom du projet	LumiQuality AI
Sous-titre	Système intelligent de prédiction de la qualité dans la fabrication de pièces plastiques par injection moulage
Type	Application web full-stack basée sur l'intelligence artificielle
Cadre	Mini-projet — ENSA Béni Mellal, filière Transformation Digitale Industrielle
Domaine d'application	Industrie 4.0, contrôle qualité, fabrication par injection plastique
Cas d'application	Production de lentilles optiques pour l'éclairage public urbain

1.2 Contexte général

L'éclairage public urbain représente un enjeu stratégique majeur pour les villes modernes en termes d'économies d'énergie, de sécurité routière et de réduction de la pollution lumineuse. Les lampadaires LED de nouvelle génération s'appuient sur des lentilles optiques en plastique injecté dont la qualité conditionne directement l'uniformité de l'éclairage sur la chaussée.

La qualité de ces lentilles est régie par la norme européenne UNI EN 13201-2:2016, qui impose une uniformité lumineuse minimale ($U_0 \geq 0.4$) pour les routes M1. Une lentille non conforme entraîne un éclairage irrégulier de la route, une non-conformité réglementaire, un coût de rebut élevé pour le fabricant, et un gaspillage énergétique.

Le procédé d'injection plastique étant influencé par de nombreux paramètres physiques (température, pression, temps, force, vitesse), la prédiction de la qualité du produit fini constitue un défi technique majeur pour les industriels. L'intelligence artificielle, et plus précisément le machine learning, offre aujourd'hui des solutions performantes pour anticiper les défauts avant le contrôle qualité final.

1.3 Problématique

Comment prédire en temps réel la classe de qualité d'une lentille plastique d'éclairage à partir des paramètres physiques collectés sur la machine d'injection pendant le cycle de production, afin d'anticiper les défauts avant le contrôle photométrique final et d'optimiser les paramètres machine ?

1.4 Objectifs

Objectif principal

Développer une application web intelligente, sécurisée et déployée, capable de prédire en temps réel la classe de qualité d'une lentille plastique à partir des 13 paramètres collectés sur la machine d'injection.

Objectifs spécifiques

- Entraîner un modèle de machine learning performant (F1-score macro > 0.80) sur des données industrielles réelles
- Comparer plusieurs algorithmes de classification (Random Forest, XGBoost, KNN, SVM) et sélectionner le meilleur
- Offrir une interface web moderne, sécurisée et accessible aux opérateurs de production
- Mettre en place une authentification professionnelle avec gestion des rôles (Keycloak)
- Stocker l'historique des prédictions dans MongoDB pour permettre l'analyse et l'amélioration continue
- Fournir un dashboard analytique pour les managers avec indicateurs de performance
- Déployer l'application sur le cloud pour une démo accessible publiquement

1.5 Source du dataset utilisé

Le projet s'appuie sur un dataset industriel réel publié dans une revue scientifique :

- Polenta, A.; Tomassini, S.; Falcionelli, N.; Contardo, P.; Dragoni, A.F.; Sernani, P. "A Comparison of Machine Learning Techniques for the Quality Classification of Molded Products". Information 2022, 13, 272.
- Source des données : usine iGuzzini Illuminazione (Recanati, Italie)
- Machine d'injection : Engel E-MAC 310/100
- Système MES : TIG Manufacturing Execution System
- Norme appliquée : UNI EN 13201-2:2016 (éclairage public européen)
- Repository GitHub : <https://github.com/airtlab/machine-learning-for-quality-prediction-in-plastic-injection-molding>

2. Description du dataset

2.1 Caractéristiques générales

Caractéristique	Valeur
Nombre d'échantillons	1451 lentilles produites
Période de production	5 jours répartis entre 2019 et 2020
Nombre de variables d'entrée	13 paramètres physiques
Nature des variables	Numériques continues
Variable cible	Classe de qualité (4 classes)
Type de problème ML	Classification multi-classes
Format du fichier	CSV (Comma-Separated Values)

2.2 Les 13 paramètres physiques d'entrée

Paramètre	Unité	Description
Melt temperature	°C	Température du polymère avant injection
Mold temperature	°C	Température du moule
Filling time	s	Temps de remplissage du moule
Plasticizing time	s	Temps de plastification du produit
Cycle time	s	Temps total du cycle de production
Closing force	N	Force de fermeture du moule
Clamping force peak value	N	Pic de force de serrage du moule
Torque peak value current cycle	N·m	Pic de couple sur la vis d'injection
Torque mean value current cycle	N·m	Couple moyen sur la vis d'injection
Specific back pressure peak	Bar	Pic de pression dorsale (résistance vis)
Specific injection pressure peak	Bar	Pic de pression d'injection
Screw position end of hold	cm	Position de la vis en fin de maintien
Shot volume	cm ³	Volume injecté

2.3 Variable cible : 4 classes de qualité

La qualité d'une lentille est mesurée par son uniformité photométrique (U0) selon la norme UNI EN 13201-2:2016. Le dataset définit 4 classes de qualité :

Label	Classe	Critère U0	Description	Échantillons
1	Rebut (Waste)	$U0 < 0.4$	Non conforme à la norme, à jeter	370
2	Acceptable	$0.4 \leq U0 < 0.45$	Conforme mais qualité limite	406
3	Cible (Target)	$0.45 \leq U0 \leq 0.5$	Qualité optimale recherchée	310
4	Inefficient	$U0 > 0.5$	Surqualité, gaspillage de ressources	360

Total : 1451 échantillons, dataset relativement équilibré entre les 4 classes.

3. Description fonctionnelle

3.1 Acteurs du système

Acteur	Rôle Keycloak	Permissions principales
Opérateur qualité	operator	Saisie des paramètres, consultation de ses prédictions, dashboard personnel
Manager	manager	Toutes les permissions opérateur + statistiques globales, rapports, vue d'ensemble
Administrateur	admin	Toutes les permissions + gestion utilisateurs, journal d'audit, configuration système

3.2 Fonctionnalités principales

FN1 — Authentification sécurisée

- Connexion via Keycloak avec login/mot de passe
- Gestion des rôles et permissions (operator, manager, admin)
- Tokens JWT pour la sécurité des appels API
- Déconnexion automatique après inactivité
- Possibilité de SSO (Single Sign-On)

FN2 — Prédiction en temps réel

- Saisie des 13 paramètres de production via un formulaire ergonomique
- Validation des données côté client (React) et serveur (Node.js)
- Prédiction instantanée par appel au modèle ML (FastAPI)
- Affichage clair du résultat avec la classe prédite (Rebut/Acceptable/Cible/Inefficient)
- Affichage des probabilités pour chaque classe
- Recommandations en cas de risque de défaut
- Indicateur visuel coloré selon la classe (rouge/orange/vert/jaune)

FN3 — Dashboard analytique

- Vue d'ensemble des indicateurs clés (KPI) du jour, de la semaine, du mois
- Nombre de lentilles produites, répartition par classe de qualité
- Taux de rebut et coût estimé
- Graphiques d'évolution dans le temps (Recharts)
- Identification des paramètres les plus influents (feature importance)

- Comparaison entre opérateurs (pour les managers)

FN4 — Historique des prédictions

- Tableau paginé filtrable par date, opérateur, classe
- Recherche par mots-clés et tri par colonne
- Export en CSV pour analyse externe
- Modal de détail d'une prédiction avec tous ses paramètres
- Modification ou suppression (selon les droits)

FN5 — Gestion des utilisateurs (admin)

- CRUD utilisateurs (création, consultation, modification, suppression)
- Attribution des rôles Keycloak
- Réinitialisation de mot de passe
- Journal d'audit des actions importantes

FN6 — Prédiction par lot

- Import d'un fichier CSV contenant plusieurs lentilles
- Traitement en batch et affichage de la progression
- Export des résultats en CSV avec les prédictions ajoutées

3.3 Cas d'utilisation principaux

Scénario 1 — Prédiction en cours de production :

L'opérateur lance un cycle de production. Pendant que la machine produit la lentille, il saisit les 13 paramètres relevés sur le panneau de contrôle. Il clique sur "Prédire". L'application affiche en moins de 2 secondes la classe prédite. Si la classe est "Rebut", l'opérateur peut décider d'ajuster les paramètres machine avant le cycle suivant.

Scénario 2 — Analyse hebdomadaire par le manager :

Le manager consulte le dashboard chaque lundi matin. Il visualise le taux de rebut de la semaine précédente, identifie les paramètres machine qui ont le plus contribué aux défauts, et programme des actions correctives (maintenance, formation, ajustement). Il génère un rapport PDF pour la direction.

Scénario 3 — Audit qualité par l'administrateur :

L'administrateur consulte le journal d'audit pour tracer toutes les actions critiques sur la plateforme. Il vérifie l'intégrité des données et la conformité avec les processus qualité de l'usine.

4. Description technique

4.1 Architecture globale

L'application repose sur une architecture microservices moderne, conforme aux standards de l'industrie 4.0, avec séparation claire des responsabilités :

- Couche présentation : Frontend React.js (interface utilisateur)
- Couche sécurité : Keycloak (authentification, autorisation, JWT)
- Couche logique métier : Backend Node.js + Express (API REST)
- Couche intelligence : Service Python FastAPI (modèle de machine learning)
- Couche données : MongoDB Atlas (prédictions, logs) + PostgreSQL (Keycloak)

4.2 Stack technique détaillée

Frontend

Technologie	Version	Usage
React.js	18.x	Bibliothèque UI principale
Vite	5.x	Build tool moderne et rapide
React Router	6.x	Navigation entre les pages
Axios	1.x	Appels HTTP vers le backend
keycloak-js	23.x	Intégration de l'authentification Keycloak
Tailwind CSS	3.x	Framework CSS utilitaire
Recharts	2.x	Graphiques interactifs du dashboard
React Hook Form	7.x	Gestion et validation des formulaires

Backend Node.js

Technologie	Version	Usage
Node.js	20.x	Runtime JavaScript
Express.js	4.x	Framework web minimaliste
Mongoose	8.x	ODM pour MongoDB
keycloak-connect	21.x	Middleware Keycloak
jsonwebtoken	9.x	Décodage et vérification JWT

Technologie	Version	Usage
axios	1.x	Appel au service ML Python
cors / helmet / morgan	—	Sécurité et logging HTTP
dotenv	16.x	Variables d'environnement

Service ML Python

Technologie	Version	Usage
Python	3.10+	Langage principal
FastAPI	0.110+	Framework API moderne
scikit-learn	1.4+	Algorithmes de machine learning
XGBoost	2.x	Boosting performant
pandas / numpy	—	Manipulation des données
joblib	—	Sérialisation du modèle
uvicorn	—	Serveur ASGI haute performance

Bases de données et infrastructure

Composant	Rôle
MongoDB Atlas (cluster M0 gratuit)	Stockage des prédictions, audit logs, statistiques
PostgreSQL 16	Base de données utilisateurs de Keycloak
Keycloak 23.x	Serveur d'authentification (Realm dédié wheelguard)
Docker + Docker Compose	Orchestration des services en développement local
Vercel	Hébergement du frontend React (déploiement gratuit)
Render.com	Hébergement du backend Node et du service ML Python

4.3 Modèle de données MongoDB

Collection : predictions

Chaque prédiction effectuée est stockée sous forme de document JSON :

```
{ _id: ObjectId, userId: "keycloak-user-uuid", username: "youssef.operator", timestamp: "2026-05-09T14:23:00Z",
  parameters: { meltTemperature: 245, moldTemperature: 65, fillingTime: 1.8, plasticizingTime: 8.5, cycleTime: 22.3,
    closingForce: 980000, clampingForcePeak: 1020000, torquePeak: 145, torqueMean: 98, backPressurePeak: 35,
    injectionPressurePeak: 850, screwPosition: 8.2, shotVolume: 47.5 }, prediction: 3, className: "Target", probabilities: {
    rebut: 0.05, acceptable: 0.12, target: 0.78, inefficient: 0.05 }, modelVersion: "rf-v1.2" }
```

4.4 Routes API REST principales

Méthode	Endpoint	Rôle requis	Description
POST	/api/predict	operator	Effectue une prédiction
POST	/api/predict/batch	operator	Prédiction par lot CSV
GET	/api/predictions/history	operator	Historique de l'utilisateur
GET	/api/predictions/all	manager	Toutes les prédictions
GET	/api/stats/dashboard	operator	KPI personnels
GET	/api/stats/global	manager	KPI globaux
GET	/api/users	admin	Liste des utilisateurs
GET	/api/audit-logs	admin	Journal d'audit

4.5 Modèle de machine learning

Algorithmes testés

- Logistic Regression (modèle de référence)
- K-Nearest Neighbors (KNN)
- Decision Tree
- Random Forest
- Gradient Boosted Trees / XGBoost
- Support Vector Machine (SVM)

Métriques d'évaluation

- Accuracy globale
- Precision, Recall, F1-score (par classe et macro)
- Matrice de confusion 4x4
- Validation croisée stratifiée 5-fold

Objectifs de performance

- F1-score macro minimum : 0.80
- Accuracy minimum : 0.85
- Temps de prédiction : moins de 200 ms par échantillon
- Détection de la classe "Rebut" prioritaire (rappel élevé)

5. Contraintes du projet

5.1 Contraintes techniques

- Application 100% web, responsive desktop et tablette minimum
- Performance : temps de réponse inférieur à 2 secondes pour une prédiction
- Sécurité : authentification obligatoire, HTTPS en production, mots de passe hashés
- Compatibilité : Chrome, Firefox, Edge dernières versions
- Code maintenable, structuré, commenté en français
- Versioning Git obligatoire avec commits réguliers

5.2 Contraintes de planning

- Soutenance prévue à partir du 17 avril 2026 (à confirmer avec l'encadrant)
- Durée totale du développement : environ 4 semaines
- Livrables attendus : code source GitHub, application déployée, rapport, slides

5.3 Contraintes d'usage

- Application utilisable sans formation extensive
- Interface en français (langue principale)
- Accessible aux utilisateurs non-techniques (opérateurs de production)
- Documentation utilisateur claire et concise

5.4 Contraintes budgétaires

- Tous les services utilisés doivent être gratuits ou freemium
- MongoDB Atlas : cluster M0 gratuit (512 MB)
- Vercel : plan Hobby gratuit pour le frontend
- Render.com : plan gratuit pour les services backend
- Keycloak : open-source gratuit

6. Résultats attendus

6.1 Livrables techniques

Livable 1 — Modèle ML entraîné

- Notebooks Jupyter documentés (exploration, préparation, modélisation)
- Modèle sauvegardé (model.pkl) avec scaler associé
- Performance F1-score macro supérieure à 0.80 documentée
- Comparaison de 6 algorithmes ML avec graphiques

Livable 2 — Service ML Python

- Micro-service FastAPI fonctionnel exposant les endpoints
- Documentation Swagger automatique générée
- Service dockerisé pour faciliter le déploiement
- Tests unitaires sur les fonctions critiques

Livable 3 — Backend Node.js

- API REST complète avec toutes les routes spécifiées
- Intégration MongoDB Atlas opérationnelle
- Middleware Keycloak fonctionnel pour la vérification JWT
- Gestion des erreurs et logging structuré
- Validation des données entrantes

Livable 4 — Frontend React

- Interface complète avec toutes les pages spécifiées
- Design moderne, responsive et accessible
- Intégration Keycloak côté client
- Graphiques interactifs avec Recharts
- Gestion des états de chargement et des erreurs

Livable 5 — Configuration Keycloak

- Realm exporté en JSON (versionné dans Git)
- Rôles configurés : operator, manager, admin
- Au moins 3 utilisateurs de test créés
- Documentation du paramétrage

Livable 6 — Déploiement

- Application accessible publiquement via une URL
- Frontend déployé sur Vercel
- Backend déployé sur Render.com
- Service ML déployé sur Render.com
- Docker Compose fonctionnel pour le développement local

6.2 Livrables documentaires

Livable 7 — Code source GitHub

- Repository public ou privé bien structuré
- README.md détaillé avec captures d'écran
- Instructions d'installation complètes
- License Open-Source (MIT recommandé)

Livable 8 — Rapport de projet

- Document de 30 à 50 pages couvrant tout le projet
- Sections : introduction, contexte, problématique, état de l'art, méthodologie, conception, implémentation, résultats, conclusion, annexes
- Captures d'écran de l'application
- Diagrammes d'architecture et UML
- Résultats du modèle avec graphiques

Livable 9 — Présentation de soutenance

- Slides PowerPoint de 12 à 15 diapositives
- Démo live préparée et répétée
- Scénarios de test pour le jury
- Plan B en cas de problème technique pendant la démo

6.3 Résultats fonctionnels attendus

À la fin du projet, l'application LumiQuality AI doit être capable de :

- Permettre à un opérateur de se connecter en moins de 5 secondes via Keycloak
- Prédire la classe de qualité d'une lentille en moins de 2 secondes après saisie des 13 paramètres
- Afficher le résultat clairement : classe (1-4), libellé, probabilités par classe, recommandations

- Stocker chaque prédiction dans MongoDB avec horodatage et identifiant utilisateur
- Fournir un dashboard à jour avec les indicateurs de qualité du jour, de la semaine, du mois
- Permettre à un manager de visualiser les statistiques globales et d'identifier les paramètres critiques
- Permettre à un admin de gérer les utilisateurs et de consulter le journal d'audit
- Fonctionner en production via une URL publique pour la démo de soutenance

6.4 Indicateurs de succès du projet

Indicateur	Cible	Méthode de mesure
F1-score macro du modèle	> 0.80	Validation croisée 5-fold
Accuracy globale	> 0.85	Test set indépendant
Temps de réponse moyen	< 2 secondes	Tests de charge avec Postman
Taux de complétude des FN principales	100%	Checklist fonctionnelle
Application déployée	URL accessible	Tests utilisateur final
Code documenté	README + commentaires	Revue de code
Rapport et slides	Livrés à temps	Date de soutenance

6.5 Bénéfices attendus pour l'industriel utilisateur

Une fois déployée dans une vraie usine d'injection plastique, LumiQuality AI permettrait de :

- Réduire le taux de rebut de 30 à 50% par détection précoce des défauts
- Optimiser les paramètres machine grâce à l'analyse des prédictions
- Réduire les coûts de production et la consommation énergétique
- Améliorer la traçabilité qualité (norme ISO 9001 / IATF 16949)
- Faciliter la formation des nouveaux opérateurs
- Constituer un historique exploitable pour l'amélioration continue

7. Workflow de réalisation

7.1 Phases du projet

Phase	Durée	Description
1. Préparation environnement	2 jours	Installation outils, GitHub, MongoDB Atlas
2. Modèle ML et service Python	4 jours	Notebooks, entraînement, FastAPI
3. Configuration Keycloak	2 jours	Docker, realm, rôles, utilisateurs
4. Backend Node.js + Express	4 jours	API REST, MongoDB, middleware Keycloak
5. Frontend React.js	5 jours	Pages, composants, intégration Keycloak
6. Intégration et tests	2 jours	Tests end-to-end, correction bugs
7. Déploiement cloud	2 jours	Vercel, Render, configuration production
8. Documentation et soutenance	3 jours	Rapport, slides, démo, répétition

Durée totale estimée : environ 24 jours de travail effectif (3 à 4 semaines).

7.2 Méthodologie de gestion de projet

- Approche agile avec livrables intermédiaires testables
- Versioning Git avec commits fréquents et messages clairs
- Tests progressifs après chaque phase
- Documentation continue en parallèle du développement
- Plan B prévu pour chaque composant critique (Keycloak notamment)

8. Conclusion

Le projet LumiQuality AI constitue une application complète et cohérente d'intelligence artificielle au service de l'industrie 4.0. En s'appuyant sur un dataset industriel réel (iGuzzini Illuminazione) et sur une architecture moderne (React, Node.js, MongoDB, Keycloak), il répond à un besoin concret du secteur de la fabrication par injection plastique : prédire la qualité d'un produit avant le contrôle final pour anticiper les défauts.

Le projet couvre l'ensemble des compétences attendues d'un ingénieur en transformation digitale industrielle :

- Maîtrise des algorithmes de machine learning et de leur évaluation
- Conception et développement d'une architecture microservices
- Intégration de technologies modernes (frontend, backend, bases de données)
- Sécurisation d'une application avec un système d'authentification professionnel
- Déploiement cloud et orchestration de services
- Gestion de projet, documentation et communication

Au-delà de l'aspect académique, l'application possède un réel potentiel industriel : elle pourrait être adaptée à de nombreux contextes de production (automobile, électronique, médical, packaging) en réentraînant simplement le modèle avec les données spécifiques de chaque usine.

Document préparé dans le cadre du mini-projet de fin de semestre

Filière Transformation Digitale Industrielle — ENSA Béni Mellal

Année universitaire 2025-2026